LLM Web Application Assessment

recon like an agent · **prove like a pro** · **never a guess**

An evidence-disciplined web / API pentest agent that works from a single URL

AGENDA

What this decks covers

01	**The trust problem**	why automated offense fails — noisy scanners vs. hallucinating agents
02	**What VERDICT is**	one URL → a full, unattended, evidence-backed assessment
03	**Three differentiators**	evidence discipline · autonomous exploration · sessions that survive
04	**Making it real**	scanning behind login, live observability, Burp integration
05	**The numbers**	two evaluation axes; XBOW, Web Security Academy, Juice Shop
06	**The frontier**	where trustworthy autonomous offense goes next

THE PROBLEM

Automated offense forces a bad trade-off

Classic scanners (DAST)

- **Fast, broad — but you can't trust the output.**
- Drowning in false positives; every hit needs a human.
- No business logic, no multi-step chains.
- Heavy per-target config; blind behind login.

Raw LLM agents

- **Autonomous, adaptive — but you can't trust the output.**
- Hallucinate findings that don't reproduce.
- No proof — you can't hand the report to a client.
- One good run, one confident-but-wrong run.

The unmet need explore like an agent, and **prove every finding like a scanner-plus-analyst.**

Both trust problems have the same fix: evidence.

WHAT IT IS

One URL in. A finished, evidence-backed assessment out.

Survey › Methodology › Diagnosis › Evidence › Report

No seed endpoint list. No test plan. No human in the loop.

How the agent stays honest

- **Claude-led, staged orchestration**
- one bounded tool call per stage - the model can't skip the work.
- **Tools gated per stage**
- survey maps; diagnosis probes; nothing off-script.
- **Model tiering**
- Opus on high-value screens, Sonnet on the rest.

What it actually does

- **Maps the app** SPA routes, XHR/fetch, forms, API shapes.
- **Signin & goes deeper** scans behind auth, compares roles.
- **Probes & proves** negative control + repeated replays.
- **Writes the report** every finding ships its own evidence.

ARCHITECTURE

A substrate built to be trusted - and inspected

Two modes, one contract

- **Pilot** — Claude-led, staged, adaptive.
- **Pipeline** — deterministic, granular, inspectable.
- Both write one screen-inventory contract — the single coupling point.
- Same evidence core underneath either path.

Evidence-of-record state

- Append-only, replayable event log.
- **It is the agent's working memory** and the live observability feed — one source of truth.
- Every finding carries the exact request/response that proves it.

Safety by construction

- **Scope gate on every network action.**
- Out-of-scope is declined, not attempted.
- **Auth is operator-supplied.**

DIFFERENTIATOR ①

Where today's AI agents break: staying logged in

Most autonomous web / LLM agents

- ✗ **Lose the login on context reset** - long runs drift to the public surface.
- ✗ **Wander into logout / delete** and destroy their own session.
- ✗ **Re-authenticate constantly**, or fail behind the wall entirely.
- ✗ **Crawl naively behind login** → the state explodes.
- ✗ **A token / usage limit ends the run** - progress lost.
- ✗ **Choke on SSO / SAML / CAPTCHA** - handed static creds, they never reach the app.

VERDICT - sessions that survive

- ✓ **Persistent auth-bearing browser profile** - survives every stage and a resume.
- ✓ **Never probes logout** - keepalive keeps the session warm.
- ✓ **Survey can't "finish" unauthenticated** - no silent public-only map.
- ✓ **Replays the authed request itself** - scans behind login, no crawl explosion.
- ✓ **Token limit → resumable pause** - in-flight work preserved.
- ✓ **Clears real login flows** - SSO, MS SAML, CAPTCHA handoffs, not just a form POST.

The bugs that matter live behind the login.
Sustaining a real authenticated session - unattended, for hours, without wrecking it - is the differentiator.

DIFFERENTIATOR ②

From one URL, it explores the way a human would

The recon a human does by hand

- **Maps an unknown app** — SPA virtual routes, XHR/fetch, form actions.
- **Infers API shapes** from captured traffic and scripts.
- **Prioritizes** high-value screens for the deep model.
- **Traverses auth** logs in, then compares roles for authz gaps.

The part scanners can't do

- **No seed list, no config, no test plan.**
- Self-directs across a surface it has never seen.
- Chains steps: recon → hypothesis → probe → proof.
- Reasons about business logic, not just signatures.

This is the moat. Detection is table stakes — many tools find a known bug.
Driving a full assessment from a single URL, unattended, is the hard part.

DIFFERENTIATOR ② · SURFACE DISCOVERY

Many ways to find every screen - not just links

Most tools follow `<a href>` and stop. VERDICT fuses many signals into one map of the attack surface.

Link graph

follow & absolutise links, depth-bounded

XHR / fetch intercept

every API the page calls, at the network layer

SPA virtual routes

client-side hash / pushState routes

JS static analysis

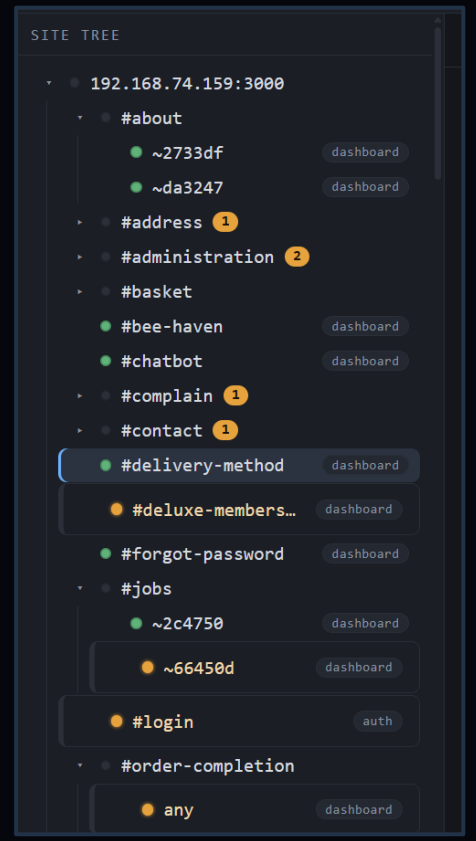
fetch / axios pulled from scripts

Path probing

guesses admin / api / backups / …

Spec ingest

OpenAPI / Swagger endpoints, no UI



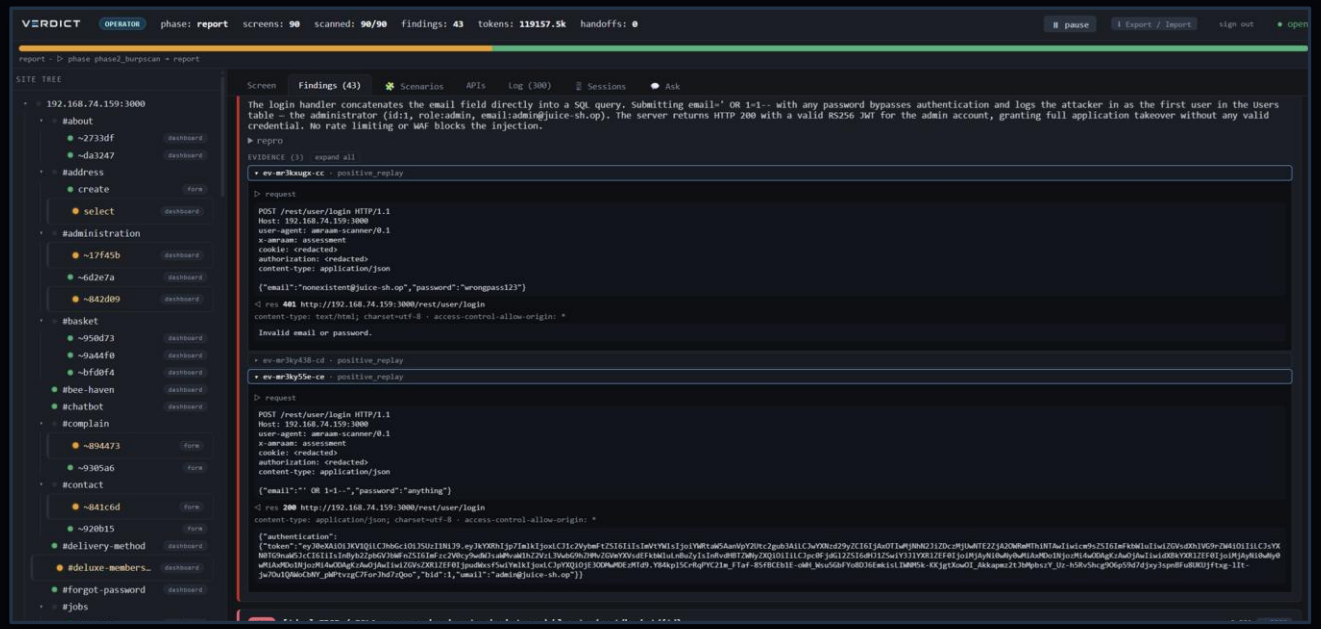
VERDICT WebUI – Site Craw

A link-crawl alone misses SPA routes, JS-only endpoints, form targets, and everything behind the login.
Fusing every signal then collapsing duplicates by DOM-skeleton hash - is how the map gets complete and clean.

DIFFERENTIATOR ③ · IN THE UI

Every finding ships the proof, inline

- Open any finding → the exact request and response that proved it.
- Negative control + positive replays, side by side.
- **The evidence is the finding.**
- Reproducible by a human in one copy-paste.



VERDICT WebUI — inline evidence viewer

Findings are confirmed only through this same negative-control + positive-replay contract — never marked by hand.

DIFFERENTIATOR ③

Confirmed means proven - not “the model thinks so”

> **The rule:** a finding is **confirmed** only with a **negative control that fails** + **≥2 positive replays that succeed**.
>
> Everything else catch-all 200s, 0-byte bodies, unstable responses - is auto-refuted. Short of proof, it files a lead, never a fabrication.

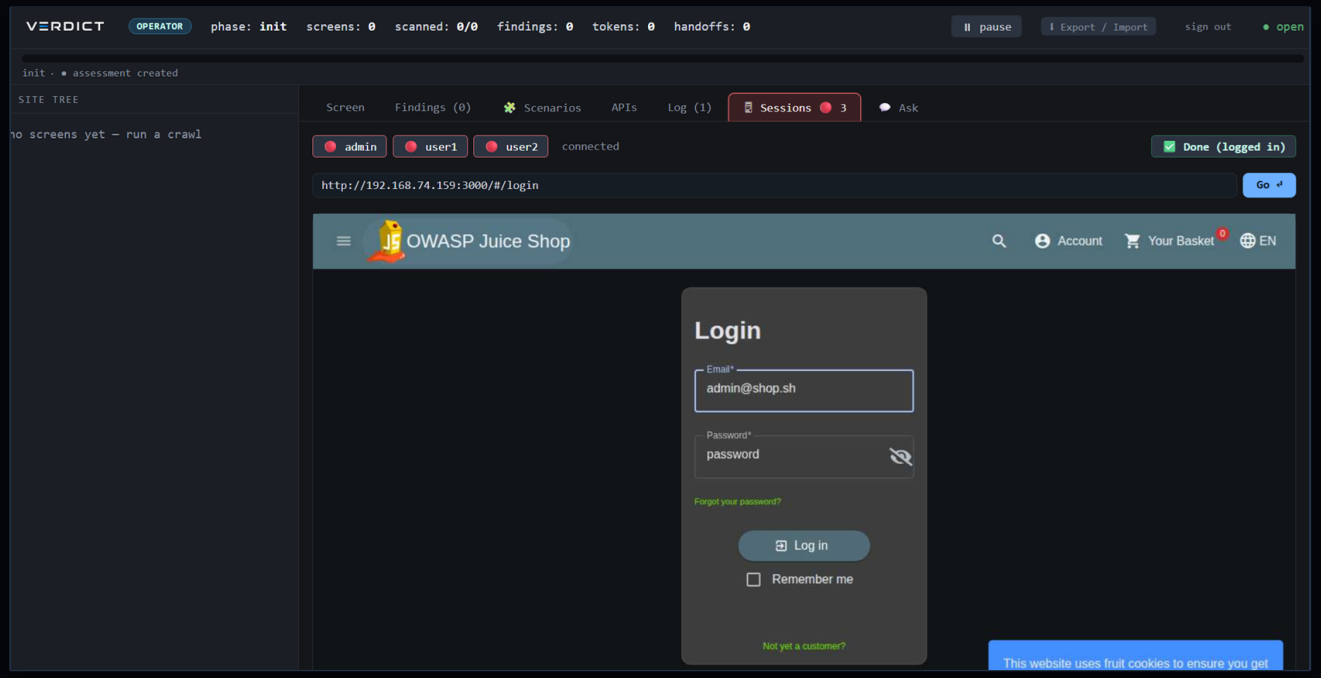
```
recorded evidence — critical SQLi auth-bypass, finding #1 of 38 (OWASP Juice Shop)

✗  negative control    {"email":"nonexistent@juice-sh.op","password":"wrong"}
                    →  401  “Invalid email or password”
✓  positive replay 1   {"email":"' OR 1=1--","password":"x"}
                    →  200  authenticated session as admin
✓  positive replay 2   same payload → 200, admin — stable, reproduced
```

This is the unlock.
Evidence discipline is what turns an autonomous LLM agent from **plausible** into **trustworthy**.

CAPABILITY · BEHIND THE LOGIN

It scans where scanners can't - past auth

- Operator supplies credentials or a captured cookie never the agent.
- Auto-discovers the login form and signs in.
- SSO / MFA walls → a human logs in once, the agent continues.
- **Multiple roles drive cross-role authorization diffing.**



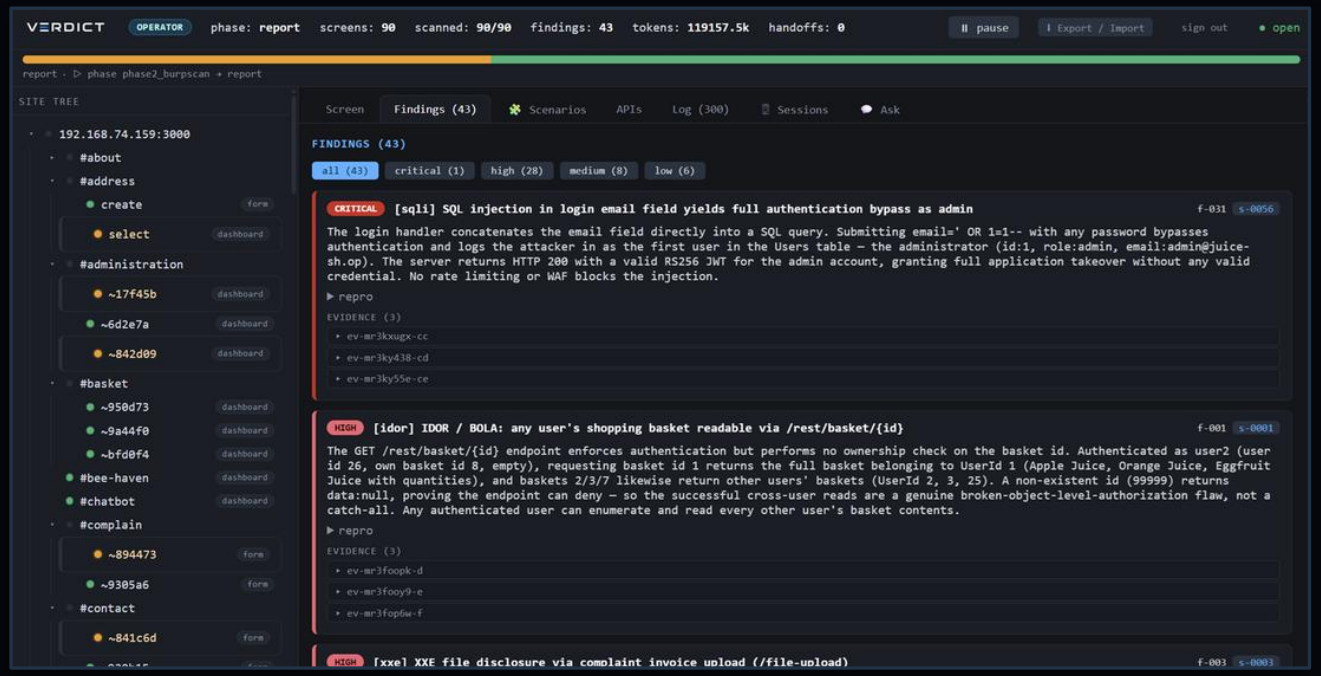
WebUI — Auth Sessions UI (Launch a separate browser for each engagement)

Auth material is operator-provided and treated as a secret; the agent uses only what it is given.

CAPABILITY · OBSERVABILITY

Watch it work - and audit every step

- A live 3-pane observer: site tree, progress, findings.
- The UI is a pure projection of the append-only event log.
- Findings, APIs, and the diagnosis log stream in real time.
- **Nothing happens off the record.**



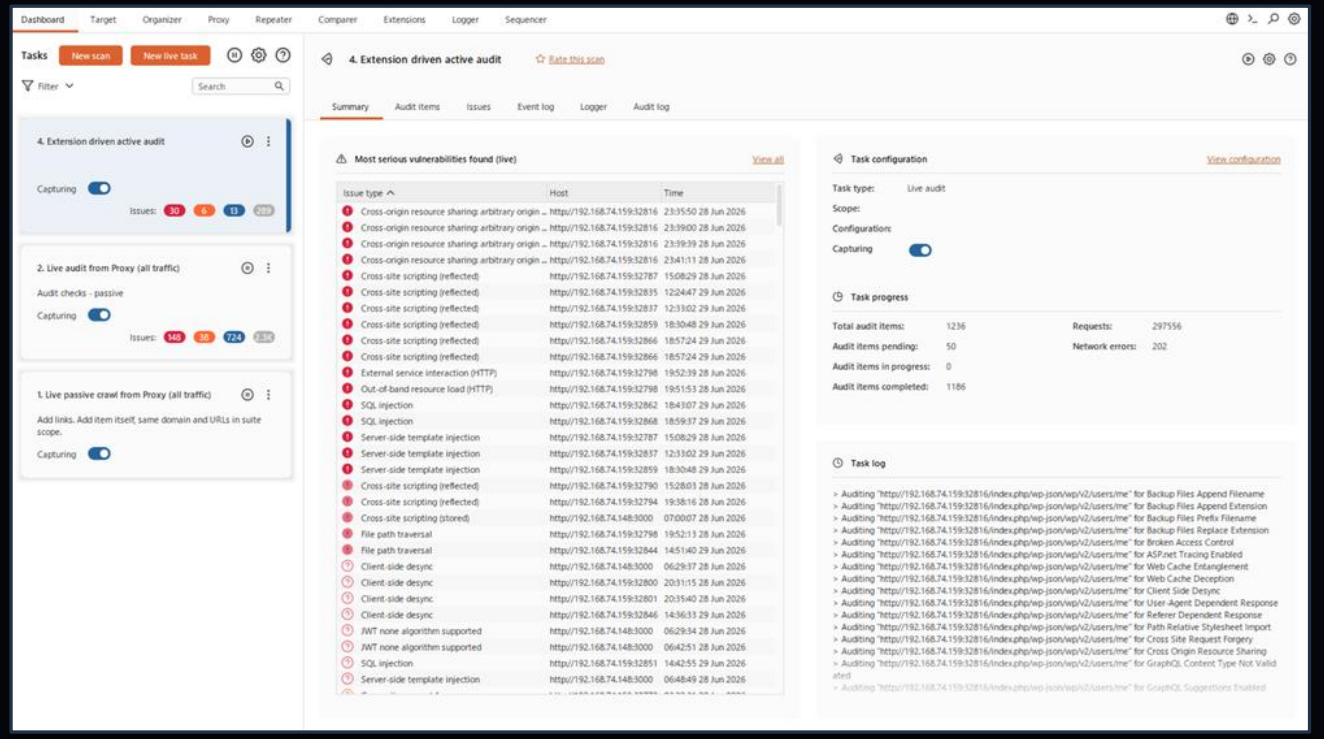
WebUI — findings pane, one page per target

Same state store powers the agent's memory and this observer — replayable end to end.

CAPABILITY · BURP

Opt-in, additive Burp integration

- Route all traffic through Burp - off by default, byte-identical when off.
- **Scan behind login: submit the authenticated request itself.**
- Out-of-band confirmation via Collaborator (blind SSRF/XXE/RCE).
- Merge net-new Burp issues, deduped against agent findings.



VERDICT-driven Burp audit (session-in-request)

Burp augments the agent; the evidence contract still gates every confirmation.

EVALUATION

Measured on two axes - because they are two skills

① Detection accuracy

› Small targets, known answer → a precise hit-rate.

› **"Can it find the vuln — even through a defense?"**

› **XBOW-Bench** — breadth × hit-rate across the class spectrum.

› **Web Security Academy** — the hardest tiers, through real defenses.

② Autonomous exploration

› One URL, unknown surface → coverage & self-direction.

› **"How much does it find on its own?"**

› **OWASP Juice Shop** — a large app, mapped and exploited unattended.

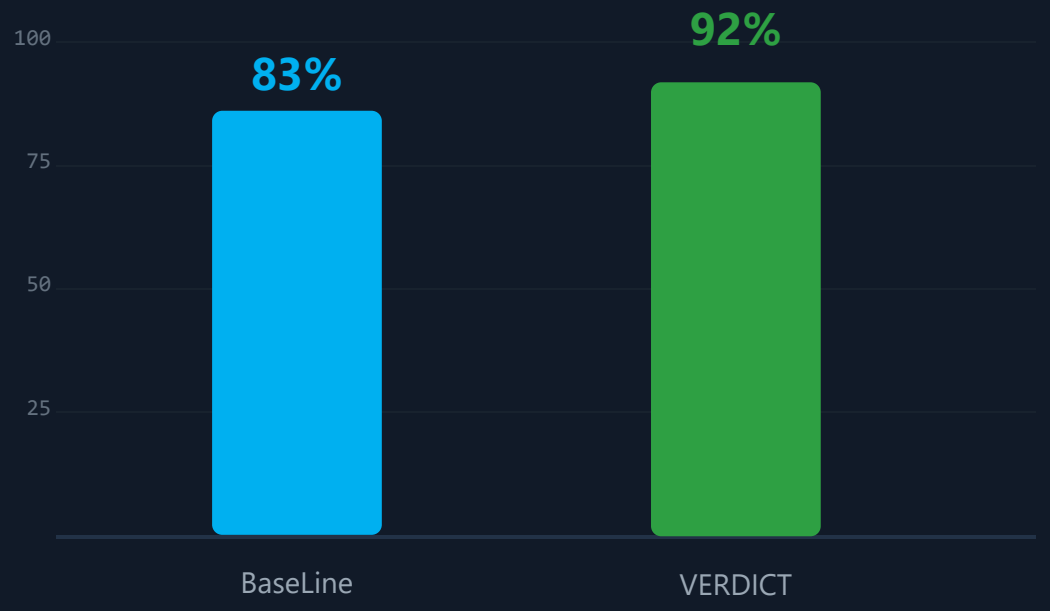
› **Live bug-bounty** — the same, on real targets.

Keeping them separate is what makes the numbers mean something — **a tool can ace one and fail the other.**

PERFORMANCE ① · XBOW-BENCH

XBOW-Bench: 92% pwn,

XBEN-24 — pwn rate over three build iterations



104 benchmarks · each iteration added a general primitive

100/109
pwned
whole XBEN-24 suite

91/91
unaided
100% with no hint

How it climbed

› Each build added a reusable exploitation primitive:

- multipart upload, blind SQLi, an XSS filter-bypass corpus,
- command injection, IDOR/BOLA verification, path traversal,
- behind-login injection, tech-aware fingerprint-then-plan.

PERFORMANCE ① · XBOW-BENCH

Broad across classes; difficulty barely moves it

93%
Level 1

91%
Level 2

89%
Level 3

23 vulnerability classes covered

- Difficulty rank barely moves the needle — what it can't do
- isn't “hard”, it's a handful of specific setups.
- **XSS · CMDi · SSRF · XXE · BSQLi · JWT · GraphQL: 100%.**
- Every result carries its proving request/response.

Pwn rate by class (selected)

PERFORMANCE ① · Web Security Academy

The hardest labs - detected through the defense

16/20
detected
two hardest tiers

12
confirmed
neg-ctrl + replays

4
leads
found, filed suspected

PortSwigger Web Security Academy

- Expert ×10 + Practitioner ×10 — the two hardest tiers.
- **Scored on detection, not the lab's "solved" flag**
- (the exploit server is a separate, out-of-scope host).

Found the vuln even against…

- **A strict cache-ability check** (web cache poisoning).
- **An AngularJS sandbox + CSP** (client-side template injection).
- **An HMAC-signed cookie** (insecure deserialization).
- **OOB-only blind XXE** (external-DTD callback).
- **A CSRF token not tied to the session** (accepted a foreign token).

PERFORMANCE ② · OWASP-Juice Shop

One unattended run. 38 proven findings.

90
Discovery Screen
all survey

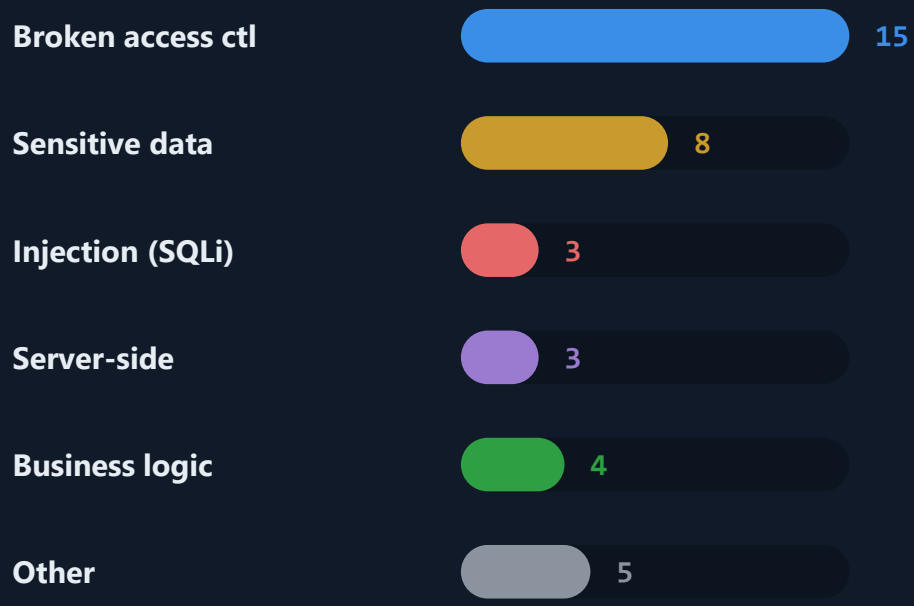
43
Findings
all evidence-backed

16
vulnerability classes
breadth + depth

OWASP Juice Shop — nobody told it where to look

- SQLi auth-bypass to admin; BOLA/IDOR across the REST API.
- **Business-logic fraud** — negative-quantity checkout, self-credit wallet.
- **Server-side** — XXE file read, SSRF with readback, null-byte traversal.
- **Sensitive data** — password hashes, a BIP39 wallet seed in public data.

Confirmed findings by area



+ confirmed findings on live bug-bounty

FRONTIER

Where trustworthy autonomous offense goes next

Scale

- Parallelize reasoning across the attack surface.
- Drive large, sprawling apps in a single engagement.

Coverage benchmarks

- Large, auth-gated, multi-role targets.
- Scored on recall + authz traversal, not just per-vuln hits.

Spec-driven APIs

- Seed from OpenAPI / Swagger.
- Assess every declared endpoint no UI required.

Closing the loop

- Scope-gated exploit delivery.
- Out-of-band confirmation (Collaborator).

The research question

How far can evidence-disciplined autonomy go
finding, and proving, real vulnerabilities with no human in the loop?

TAKEAWAYS

Three things to leave with

01

Evidence discipline is the unlock.

A negative control + repeated replays is what makes an autonomous LLM agent trustworthy instead of merely plausible. It's the whole design.

02

Autonomy + proof is a new tool.

Not a scanner (no logic, needs config) and not a chatbot (no proof). It recons like an agent and evidences like an analyst — from one URL.

03

Measured, not asserted.

92% on XBOW, 16/20 on the hardest PortSwigger tiers, 38 proven findings on Juice Shop — reproducible, with the evidence embedded in every finding.

LLM Web Application Assessment

It finds real vulnerabilities from a Top URL - and proves every one.

Reproducible benchmarks · evidence embedded in every finding · authorized targets only